

Academic Agent Platform — Database Design

Schema specification, design rationale, and deployment guidance

Table of Contents

Part A — Design principles.....	4
A.1 One data layer, not 72 application databases.....	4
A.2 Course identity and course version are different entities.....	4
A.3 <code>course_offering</code> is the central grain.....	4
A.4 Registration is separate from section membership.....	4
A.5 Question-level marks are stored, not just totals.....	4
A.6 Derived analytical results are stored, not recomputed.....	5
A.7 Every consequential record carries its provenance.....	5
A.8 Access is enforced at the data layer.....	5
A.9 Confidential data lives in a separate schema.....	5
A.10 Configuration is data, not code.....	5
A.11 Prohibited model inputs are declared, not merely omitted.....	5
Part B — Schema map.....	6
Foundation.....	6
Academic core.....	6
Research and engagement.....	6
Student lifecycle.....	7
Institutional.....	7
Platform.....	7
Part C — The core spine.....	8
Part D — The parts that are easy to get wrong.....	9
D.1 Regulation versioning.....	9
D.2 Attendance grain and denominators.....	9
D.3 The evidence chain.....	9
D.4 Partial payments and the fee waterfall.....	9
D.5 Publication deduplication.....	10
D.6 Venue metrics are time-stamped.....	10
Part E — The agentops schema.....	11

E.1 Registry and constraint.....	11
E.2 Run provenance.....	11
E.3 The human approval gate.....	11
E.4 The detect–decide–act–measure loop.....	11
E.5 Model governance.....	12
Part F — Security model.....	13
F.1 Roles and scope.....	13
F.2 Session context.....	13
F.3 Row-level security.....	13
F.4 Separation and immutability.....	14
F.5 Data protection readiness.....	14
Part G — Physical design.....	15
G.1 Volume estimates.....	15
G.2 Partitioning.....	15
G.3 Indexing.....	16
G.4 Vector search.....	16
G.5 Analytics separation.....	16
Part H — Data ingestion and quality.....	17
H.1 The ingestion gate.....	17
H.2 Document extraction confidence.....	17
H.3 Identity resolution.....	17
H.4 Anomaly capture.....	17
Part I — Build order.....	18
Phase 0 — Foundation.....	18
Phase 1 — Retrieval agents.....	18
Phase 2 — Core analytics.....	18
Phase 3 — Operations and research.....	18
Phase 4 — Predictive and composite.....	19
Phase 5 — Remaining domains.....	19
A recommended starting subset.....	19
Part J — Deliverables and how to use them.....	20
J.1 Files.....	20
J.2 Deployment.....	20
J.3 What is deliberately not included.....	21

J.4 Open decisions for the institution.....	21
Appendix — Table inventory.....	22

This document specifies the database underlying the 72-agent platform. It accompanies a set of PostgreSQL DDL files that have been executed and tested: 221 tables across 21 schemas, 457 foreign keys, 431 indexes, and 19 row-level security policies. Everything described here runs.

The document covers the design decisions and their reasons, the schema map, the parts that are easy to get wrong, the security model, physical design guidance, and a phased build order. An appendix lists every table.

Part A — Design principles

Eleven decisions shape the whole model. Most of the cost of getting a schema like this wrong is concentrated in these.

A.1 One data layer, not 72 application databases

Every agent reads and writes through one canonical model. The alternative, where each agent owns its own store, produces 72 inconsistent answers to the same question within a year. Pass percentage must mean one thing.

A.2 Course identity and course version are different entities

A course has a stable identity: its title, its owning department. It also has a regulation-specific version carrying credits, contact hours, syllabus units, and course outcomes. `curriculum.course` holds the first; `curriculum.course_version` holds the second.

This is not fussiness. A typical institution has three or four regulations running simultaneously across live batches. A 2022-batch student and a 2025-batch student taking a course with the same name are taking different courses with different credits and different outcomes. Collapsing them into one row is the single most common cause of wrong answers in academic systems, and it is very expensive to unpick later.

A.3 `course_offering` is the central grain

One `course_version`, in one term, taught to one section. Allocation, timetable, lesson plan, class sessions, attendance, assessments, marks and outcome attainment all hang off this.

If this grain is wrong or ambiguous, every downstream number is ambiguous. It is worth spending time confirming that this definition matches how the institution actually operates, including edge cases such as merged sections for electives and courses taught jointly by two departments.

A.4 Registration is separate from section membership

`academics.student_registration` records who is actually taking an offering. Section membership is not sufficient, because electives, backlog re-registration, lateral entry, and audit registrations all break the assumption. Attendance denominators and result populations both depend on this table being right.

A.5 Question-level marks are stored, not just totals

`assessment.student_question_mark` records each student's mark on each question, and `assessment.paper_question` tags each question to a course outcome, unit and Bloom level.

Course outcome attainment cannot be automated without this. Institutions that capture only assessment totals discover during accreditation that a year of evidence must be

reconstructed by hand from physical answer scripts. Capturing question-level marks costs almost nothing at entry time if the mark entry screen is designed for it from the start.

A.6 Derived analytical results are stored, not recomputed

`attendance.attendance_summary`, `academics.coverage_snapshot`, `hr.faculty_workload` and `quality.kpi_value` are tables, not views. Each row carries `as_of_date` and the agent that computed it.

The reason is that the history of a derived figure is itself the reportable artefact. Knowing that a student's attendance was 82 percent in week six and 70 percent in week ten is what makes the trend and the projection possible. A view that always recomputes from current data cannot answer what the position was last month.

A.7 Every consequential record carries its provenance

`faculty_allocation` stores the match score and rationale the optimiser produced, alongside the human who approved it and any override reason. `co_attainment` stores the specific question ids that contributed. `agent_output` stores the reasoning summary and citations. `risk_flag` stores the contributing signals.

This is what allows a peer team, a contesting faculty member, or an auditor to ask why the system said what it said, and get an answer.

A.8 Access is enforced at the data layer

Row-level security policies on the sensitive tables mean an agent is technically incapable of retrieving another student's record, regardless of how the question is phrased. Prompt-level instructions are not an access control mechanism.

A.9 Confidential data lives in a separate schema

Counselling, crisis escalation, and health-related records are in `confidential`, with its own grants and its own encryption. The general application roles have no access at all. Disciplinary records sit in `studentlife` but under their own RLS policy and with a mandatory `retention_until` date.

Aggregation creates risk that the individual source systems did not carry. The 360-degree student profile view deliberately excludes all of this.

A.10 Configuration is data, not code

Attainment rubrics, workload norms, fee structures, appraisal rubrics, scholarship eligibility criteria, and code lists are all rows. Institutions revise these between cycles, and different programmes within one institution frequently use different values. Hard-coding any of them guarantees a change request every year.

A.11 Prohibited model inputs are declared, not merely omitted

`agentops.model_version.excluded_features` records which attributes a predictive agent must not use. Declaring the exclusion means that a later change reintroducing a protected attribute, or a proxy for one, is visible in review rather than invisible in a feature list.

Part B — Schema map

Twenty-one schemas, grouped by function.

Foundation

Schema	Purpose	Agents served
core	Institution, campus, department, academic year, term, calendar, rooms, code lists	All
people	Person supertype; student, faculty, staff, guardian subtypes; identity resolution	All
identity	Users, roles, permissions, scoped role assignment, delegation, audit log, consent, data subject requests	All

`people.person` is a supertype. One human being, one row, regardless of whether they are simultaneously a student, an alumnus, and a guest lecturer. `people.person_identifier` maps them to their identifiers in each source system, which is where identity resolution becomes auditable rather than a spreadsheet exercise.

Academic core

Schema	Purpose	Agents
curriculum	Programme, regulation, batch, section, course, course version, units, topics, COs, POs, CO-PO map, books	1, 2, 8, 31, 54, 67
academics	Course offering, registration, faculty allocation, timetable, lesson plan, class sessions, topic progress, coverage	3, 4, 5, 6, 7
attendance	Attendance records (partitioned), student leave, summaries, condonation, ingestion validation	11, 16
assessment	Assessment types, question bank, blueprints, papers, question marks, internal marks, results, backlogs	12, 15, 31, 32, 33, 34, 35
exams	Exam events, schedules, eligibility, seating, invigilation, scripts, evaluation, revaluation, malpractice	30
outcomes	Attainment rubrics, runs, CO attainment, PO attainment, indirect feedback, action taken	8, 9

Research and engagement

Schema	Purpose	Agents
research	Venues, venue metrics, publications, authorship, citations, profiles, patents, funding calls, proposals, projects, doctoral scholars, collaborations	17–25
engagement	Events, participants, submissions, reviews, industry partners, MoUs and deliverables, industry activity, outreach	26–29

Student lifecycle

Schema	Purpose	Agents
admissions	Enquiries, interactions, cut-off history, applications, documents, counselling, allotment, enrolment	36–39
finance	Fee heads, structures, demands, payments, allocations, refunds, reminders, scholarships, loan documents	40–43
studentlife	Mentoring, meetings, action items, grievances, discipline, achievements, certifications	44–48
placement	Companies, openings, drives, offers, readiness, internships, alumni, contributions	49–52
confidential	Counselling cases and notes, crisis escalation, academic accommodations	66

Institutional

Schema	Purpose	Agents
hr	Workload norms, workload computation, appraisal, development plans, leave, substitutions, faculty attendance	58–61
governance	Policy documents and clauses, circulars, committees, meetings, compliance frameworks and checks	53–57
quality	KPI definitions and values, accreditation criteria, evidence items, readiness, feedback instruments	9, 57, 71

Platform

Schema	Purpose	Agents
knowledge	Documents, chunks, embeddings, extraction jobs, extracted fields	53, 64, 65
agentops	Agent registry, tool declarations, runs, inputs, outputs, human review, risk flags, interventions, outcomes, alerts, model versions, fairness audits, incidents, schedules	All

Part C — The core spine

Eight entities carry the structural weight. Everything else attaches to them.



Two things to note. First, `course_offering` sits at the convergence of the curriculum branch and the calendar branch, which is why it is the grain. Second, `person` is the root of all human data, which is what makes a single access control model possible.

Part D — The parts that are easy to get wrong

D.1 Regulation versioning

`curriculum.regulation` carries `effective_from_admission_year` and `effective_to_admission_year`, plus a `superseded_by` link. `course_version` is unique on `(regulation_id, programme_id, course_code)`.

Any query about a student must resolve through their batch to their regulation. A query for “the syllabus of CS301” without a regulation context is under-specified and the API should reject it rather than guess.

D.2 Attendance grain and denominators

`attendance.attendance_record` is one row per student per `class_session`. It is partitioned by `session_date`.

The denominator is the count of `class_session` rows with `status = 'CONDUCTED'`, not the timetable. Reality diverges from the timetable through cancellations, substitutions, compensation classes and extra classes, and computing attendance against the plan rather than against what happened produces figures that faculty will correctly dispute.

`attendance_summary` stores both `raw_pct` and `adjusted_pct`, the latter after applying on-duty and approved-leave rules. Every report must state which one it is using. It also stores `trend_slope` and `projected_end_pct`. The projection is the number that changes student behaviour; the current percentage only tells them where they are.

D.3 The evidence chain

Accreditation requires that a summary figure resolve to its source. The chain is:

`student_question_mark` → `paper_question` (tagged to `course_outcome`) → `co_attainment` (within an `attainment_run` bound to a rubric version) → `po_attainment` (weighted by `co_po_map.strength`) → `quality.evidence_item` → the submitted report.

The view `outcomes.v_attainment_trace` walks this in one query. When a peer team asks how a programme outcome attainment of 2.4 was arrived at, the answer should take seconds. Retrofitting this traceability is very difficult; it has to be in the model from the start.

`co_attainment.contributing_questions` stores the actual question ids, so the trail does not depend on reconstructing a join.

D.4 Partial payments and the fee waterfall

`finance.fee_head.allocation_priority` defines the order in which a partial payment is applied across heads. `payment_allocation` records where each rupee went. Without an explicit waterfall, two people computing the same student's tuition outstanding will get different answers.

`fee_demand.scholarship_expected` exists so that reminder suppression is possible. Chasing a student for fees a sanctioned scholarship already covers is the most common avoidable failure in fee follow-up, and `reminder_dispatch.suppressed` with its reason makes the suppression auditable.

D.5 Publication deduplication

The same paper arrives from Scopus, Web of Science and Google Scholar in different forms. `research.publication` holds the resolved record; `research.publication_source` holds one row per source with the raw record, the match method and the match confidence.

`research.affiliation_variant` is a small table with outsized effect. A single institution appears in the literature under a dozen spellings, and the completeness of this list determines the recall of the publication sweep more than anything else in the pipeline.

D.6 Venue metrics are time-stamped

`research.venue_metric` carries `checked_at` and `valid_until` per source per year. Indexing status changes. Presenting a cached status as current is how a faculty member publishes in a journal that was delisted last month.

Part E — The agentops schema

This is what distinguishes the platform from a reporting system, and it is the part most implementations omit.

E.1 Registry and constraint

`agentops.agent` records each agent's class (1 retrieval, 2 analytical, 3 predictive), its reasoning policy, whether it requires human approval, its escalation rule, and its named owner. `agent_tool` declares exactly which schemas and objects it may reach and under what row scope. An agent cannot reach data outside its declared tool set.

Agents without owners degrade silently, so `owner_user_id` should be treated as mandatory in practice even though the column is nullable for bootstrap.

E.2 Run provenance

`agent_run` records every invocation with its trigger, its scope, its latency and its cost. `agent_run_input` records exactly which records fed it. `parent_run_id` captures orchestration chains, so a composite answer from Agent 70 can be decomposed into the agent calls that produced it.

E.3 The human approval gate

`agent_output.requires_approval` and `approval_status`, with `human_review` recording the decision, the modified payload where applicable, and the reason.

`human_review.reason_category` is the most valuable field in the schema for improving the system. An agent whose recommendations are rejected more than half the time is telling you something specific about either its logic or its inputs, and the category tells you which.

E.4 The detect–decide–act–measure loop

`risk_flag` → `intervention` → `intervention_participant` → `intervention_outcome`.

Three design points. First, `risk_flag` stores `baseline_value` and `observed_value`, because the meaningful unit is deviation from a student's own normal, not distance from a cohort average. Second, `contributing_signals` is required, so the responder receives evidence rather than a bare flag. Third, `confirmed_by_human` is captured on closure, which is what makes the false-positive rate measurable and thresholds recalibrable.

`intervention_outcome.comparison_group_delta` is the column that separates measurement from wishful thinking. Improvement in participants means nothing without a comparable group who did not attend. Most institutions run remedial classes for years without ever establishing whether they change outcomes; this column costs almost nothing and eventually reallocates significant effort.

E.5 Model governance

`model_version` records the feature list, the explicitly excluded features, the training window, and holdout accuracy, precision, recall and false-positive rate. No Class 3 agent should reach ACTIVE without a validated model version.

`fairness_audit` is required each term per Class 3 agent per group attribute, with `published_internally` tracked, because an unpublished audit tends not to be acted on.

`agent_incident` covers what happens when an agent produces a harmful or materially wrong output, including whether affected people were notified.

Part F — Security model

Four layers.

F.1 Roles and scope

`identity.user_role` binds a user to a role with an explicit scope: `SELF`, `SECTION`, `COURSE_OFFERING`, `PROGRAMME`, `DEPARTMENT`, `CAMPUS` or `INSTITUTION`. Scope determines which rows the user sees, and it is time-boxed with `valid_from` and `valid_to`. `identity.delegation` handles temporary transfer of approval authority.

F.2 Session context

The application sets, on every connection inside a transaction:

```
SET LOCAL app.user_id      = '<uuid>';
SET LOCAL app.student_id   = '<uuid>';    -- when the user is a student
SET LOCAL app.faculty_id   = '<uuid>';
SET LOCAL app.role_codes   = 'FACULTY,MENTOR';
SET LOCAL app.dept_scope   = '<uuid>,<uuid>';
SET LOCAL app.agent_code   = 'A11_ATTENDANCE_ANALYSIS'; -- when an agent acts
```

`SET LOCAL` rather than `SET` matters: outside a transaction it does not apply, and the policies will then correctly return nothing.

F.3 Row-level security

Policies are in place on `attendance_summary`, `internal_mark`, `question_paper`, `counselling_case`, `counselling_note`, `disciplinary_case`, and `risk_flag`. Helper functions `teaches_offering()`, `is_mentor_of()` and `student_in_scope()` express the common predicates.

Verified behaviour:

Test	Result
Student A queries attendance summaries	1 row, their own
Student B queries the same table	1 row, their own
Faculty teaching the offering	Both students
Faculty not teaching it	0 rows
<code>app_readonly</code> reads confidential	Permission denied at schema level
<code>UPDATE</code> on <code>identity.audit_log</code>	Rejected, append-only

Two policies deserve specific mention. `question_paper` access is time-boxed by `access_opens_at` and `access_closes_at` in addition to role, so a setter's access expires. `risk_flag` excludes `WELLBEING_CONCERN` from the academic leadership policy entirely, routing it to the counsellor policy instead.

F.4 Separation and immutability

`confidential` is not granted to any general application role. `identity.audit_log` is partitioned by time and protected by a trigger that rejects `UPDATE` and `DELETE`.

`hr.faculty_leave_public` is a view that masks the medical reason for leave, and substitution notifications must read from it rather than the base table.

`studentlife.disciplinary_case.retention_until` is enforced in the RLS predicate itself, so an expired record becomes invisible even before the expunge job runs.

F.5 Data protection readiness

`identity.consent` records purpose-bound consent with grant and withdrawal timestamps.

`identity.data_request` handles access, correction, erasure and consent-withdrawal requests with a due date. `people.person.social_category` carries a comment restricting it to statutory reporting and fairness auditing.

Verify the current position under India's Digital Personal Data Protection Act with legal counsel rather than relying on this document; the rules and enforcement position have been developing.

Part G — Physical design

G.1 Volume estimates

For an institution of 5,000 students, 300 faculty, and 8 programmes, per academic year:

Table	Rows per year	Notes
<code>attendance.attendance_record</code>	4.5–5 million	5,000 students × ~30 periods/week × ~32 instruction weeks. The dominant table.
<code>assessment.student_question_mark</code>	1.5–2 million	5,000 × ~8 courses × ~4 assessments × ~10 questions
<code>assessment.student_assessment_mark</code>	160,000	
<code>academics.class_session</code>	25,000–35,000	
<code>academics.student_registration</code>	40,000	
<code>assessment.course_result</code>	40,000–45,000	Including supplementary attempts
<code>identity.audit_log</code>	5–20 million	Depends on how granular read logging is
<code>agentops.agent_run</code>	100,000–500,000	Scheduled sweeps dominate
<code>knowledge.chunk_embedding</code>	50,000–200,000	One-time bulk plus incremental

Total storage after five years, with indexes, is in the range of 250–600 GB. This is comfortably within a single well-provisioned PostgreSQL instance. Distributed architecture is not required and would add cost without benefit.

G.2 Partitioning

Two tables are partitioned by range on date:

- `attendance.attendance_record` by `session_date`
- `identity.audit_log` by `occurred_at`

Yearly partitions are provided for 2026 and 2027. Add partitions ahead of time, ideally through `pg_partman` or a scheduled job, because an insert with no matching partition fails.

Consider partitioning `assessment.student_question_mark` by term once it passes roughly 10 million rows.

Note that on a partitioned table the primary key must include the partition column, which is why these two tables have composite primary keys.

G.3 Indexing

431 indexes are defined. The patterns worth knowing:

- **Partial indexes on hot predicates.** `idx_backlog_open` covers only `status = 'PENDING'`; `idx_qbank_lookup` covers only validated question bank items; `idx_alert_unread` covers only unread alerts. These are much smaller and much faster than full indexes for the queries that actually run.
- **Trigram indexes for fuzzy text.** `pg_trgm` GIN indexes on `person.full_name`, `venue.name`, `publication.title`, `policy_clause.clause_text` and `document_chunk.chunk_text`. These support the name matching and clause searching that agents do constantly.
- **Deadline indexes.** `idx_patent_due`, `idx_phd_milestone_due`, `idx_call_deadline` and `idx_compliance_status` all support the scheduled sweeps that generate alerts.

G.4 Vector search

`knowledge.chunk_embedding.embedding` is declared as `real[]` so that the schema loads without extensions. In production, install `pgvector` and change it:

```
CREATE EXTENSION vector;
ALTER TABLE knowledge.chunk_embedding
  ALTER COLUMN embedding TYPE vector(1536) USING embedding::vector;
CREATE INDEX ON knowledge.chunk_embedding
  USING hnsw (embedding vector_cosine_ops);
```

At the volumes above, `pgvector` inside the same PostgreSQL instance is sufficient. A separate vector database is unnecessary complexity for a knowledge base of this size, and keeping embeddings alongside the relational data makes permission filtering on retrieval much simpler.

Retrieval must filter on `document.is_retrievable_by_agents` and on sensitivity against the requesting role, before the vector search rather than after.

G.5 Analytics separation

Operational agents read from the transactional schemas. Analytical and reporting agents, particularly 10, 20, 34, 38, 63, 71 and 72, should read from a separate read replica or a nightly-refreshed star schema.

A minimal warehouse for this platform needs conformed dimensions on student, faculty, course version, term, department, programme and section, with fact tables for attendance, assessment, results, publications, placements and finance. Grain each fact at the level its source table already uses so that the mapping stays trivial.

Do not build the warehouse in phase one. Build it when dashboard queries begin to compete with transactional load, which for an institution of this size is typically in year two.

Part H — Data ingestion and quality

H.1 The ingestion gate

`attendance.ingestion_batch` is the pattern to replicate for every bulk source. It records the source, the row counts, the validation report, and a status that must reach `ACCEPTED` before downstream analysis runs.

Agent 11 must not analyse a batch in `FAILED` or `PARTIAL` state. Silently analysing corrupt data is worse than producing nothing, because the output looks plausible.

Validation for attendance specifically: sessions recorded as conducted match the timetable or an explained variance; no student marked present in two simultaneous periods; every roll number resolves against the student master; total classes held is consistent within a section.

H.2 Document extraction confidence

`knowledge.extracted_field.confidence` gates entry into academic and financial records. Below the configured threshold the field routes to `NEEDS_REVIEW` and a human confirms it.

Optical character recognition on scanned certificates and marks cards has a material error rate, and a transposed digit in a roll number attaches one student's result to another.

`admissions.application_document.verification_status` follows the same principle: `AUTO_FLAGGED` never means rejected, and only a human sets `REJECTED`.

H.3 Identity resolution

`people.person_identifier` maps each person to their identifier in each source system, with a uniqueness constraint on `(system_code, identifier)`.

`people.person.merged_into` handles duplicate resolution without deleting history.

Expect this to be the single largest effort in phase zero. Institutions routinely find that the same student exists under different identifiers in the ERP, the biometric system and the examination system, and that reconciling them surfaces genuine data errors that have been latent for years.

H.4 Anomaly capture

`assessment.mark_anomaly` records detected problems with an explicit status lifecycle, so that an anomaly is either explained, corrected, or dismissed with a reason rather than quietly ignored. The same pattern would serve well for attendance and finance if those areas prove noisy.

Part I — Build order

The schema is modular and the files are numbered in dependency order. Deploy in phases matching the agent roadmap.

Phase 0 — Foundation

Files 01, 02, 03, 11.

Schemas: core, people, identity, curriculum, plus the security layer.

Work: integration pipelines from existing systems, identity resolution, three years of historical data cleaning, single sign-on, and the role and permission model. No agents ship in this phase, which is why it gets skipped and why skipping it is the most expensive available mistake.

Exit criterion: a validated curriculum structure for every live regulation, a resolved person register, and one demonstration query proving the pipeline works end to end.

Phase 1 — Retrieval agents

Files 09 (partial: governance, knowledge), 12.

Adds the document store, policy documents and clauses, circulars, and the read views. Supports Agents 1, 2, 53, 55, 65.

Exit criterion: students and faculty getting cited, correct answers to natural language questions.

Phase 2 — Core analytics

Files 04, 05.

Adds academics, attendance, assessment, exams, outcomes. Supports Agents 6, 8, 10, 11, 33, 34.

This is the largest single increment and the one that delivers the most operational value. Attendance analysis runs weekly and touches everyone; outcome attainment removes the largest manual burden in accreditation work.

Exit criterion: weekly automated attendance analysis with intervention lists, and course outcome attainment computed rather than compiled.

Phase 3 — Operations and research

File 06, plus hr from file 08.

Adds research, engagement, hr. Supports Agents 3, 4, 17–29, 58–61. The research block is largely independent of the student data work and can proceed in parallel with a separate team.

Phase 4 — Predictive and composite

File 10.

Adds agentops in full. Supports Agents 12, 14, 15, 16, 69, 70.

Do not attempt this before four terms of clean data exist. Every model must be validated against a held-out historical cohort, recorded in `model_version`, and audited in `fairness_audit` before it influences a real decision.

Phase 5 — Remaining domains

Files 07, remainder of 08, 09.

Adds admissions, finance, studentlife, confidential, placement, quality. Sequence these against the institutional calendar: admissions before the admission season, quality before the accreditation cycle, placement before the placement season.

A recommended starting subset

If the institution wants to prove the concept before committing, the tables needed for the six-agent minimum set are:

`core (all)`, `people.person`, `people.student`, `people.faculty`, `identity (all)`, `curriculum (all)`, `academics.course_offering`, `academics.student_registration`, `academics.faculty_allocation`, `academics.class_session`, `attendance (all)`, `assessment.assessment`, `assessment.student_assessment_mark`, `assessment.paper_question`, `assessment.student_question_mark`, `assessment.internal_mark`, `outcomes (all)`, `knowledge (all)`, `governance.policy_document`, `governance.policy_clause`, `agentops (all)`, and the views in file 12.

That is roughly 80 tables, one department, one term.

Part J — Deliverables and how to use them

J.1 Files

File	Contents
01_foundation.sql	Extensions, 21 schemas, audit trigger helper, core
02_people_identity.sql	people, identity, audit log, consent, data requests
03_curriculum.sql	Programmes, regulations, batches, sections, courses, outcomes
04_academics_attendance.sql	Offerings, allocation, timetable, delivery, attendance
05_assessment_exams_outcomes.sql	Assessment, question bank, exams, CO/PO attainment
06_research_engagement.sql	Publications, patents, funding, doctoral, events, industry, outreach
07_admissions_finance.sql	Admissions funnel, fees, scholarships, loan documents
08_studentlife_placement_hr.sql	Mentoring, grievance, discipline, confidential, placement, HR
09_governance_quality_knowledge.sql	Policy, circulars, committees, compliance, KPIs, documents
10_agentops.sql	Agent registry, runs, review, flags, interventions, model governance
11_security_rls.sql	Session helpers, RLS policies, application roles, grants
12_views.sql	Read layer: roster, attendance, performance, attainment trace, agent health
99_smoke_test.sql	End-to-end seed data proving the attainment chain works
schema_full.sql	Files 01–12 concatenated, for a single-command deployment

J.2 Deployment

```
createdb acadagents
psql -d acadagents -v ON_ERROR_STOP=1 -f schema_full.sql
```

To verify, load the smoke test and run the attainment trace:

```
psql -d acadagents -f 99_smoke_test.sql
psql -d acadagents -c "SELECT * FROM outcomes.v_attainment_trace;"
```

The smoke test creates one institution, one department, one programme under regulation R23, one course with two units and two course outcomes mapped to a programme outcome, two students, one faculty member, one offering, attendance summaries, a mid-term with question-level marks, and a computed attainment chain. It then walks a programme outcome level back to the individual question ids that produced it.

Do not load `99_smoke_test.sql` into a production database.

J.3 What is deliberately not included

No triggers computing business logic. Attendance percentages, internal marks and attainment levels are computed by the agents and written as rows, not derived by database triggers. Trigger-computed business logic is difficult to test, difficult to change, and invisible to the people who need to understand it.

No historisation on every table. Only where the history is genuinely used: derived analytical snapshots, regulation and policy versioning, mark change logs, and the audit trail. Universal temporal tables would double the model's size for little benefit.

No warehouse star schema. Deferred to phase two or later, as described in G.5.

Sizing of text columns. PostgreSQL text has no performance penalty over `varchar(n)`. Where a business rule genuinely constrains length, add a CHECK.

J.4 Open decisions for the institution

Five things the schema accommodates but cannot decide:

1. **Internal mark formula.** `assessment.internal_mark.formula_version` and components support any formula. The actual rule, including best-of-two handling and rounding, needs a written specification before Agent 33 is built.
2. **Attainment rubric.** Threshold, level bands and direct/indirect weighting go in `outcomes.attainment_rubric`. Different programmes may need different rubrics.
3. **Workload weights.** `hr.workload_norm.activity_weights` needs the institution's actual conversion factors for lab hours, project supervision and administrative roles.
4. **Audit log granularity.** Logging every read produces 20 million rows a year and genuine forensic capability. Logging only writes and sensitive reads produces a fraction of that. Decide deliberately.
5. **Retention periods.** `disciplinary_case.retention_until` has no default. Set an institutional policy rather than leaving it to whoever creates the record.

Appendix — Table inventory

Generated from the deployed database. 221 base tables plus 4 partitions.

academics (12 tables)

Table	Cols
class_session	17
course_offering	12
coverage_snapshot	12
faculty_allocation	18
lesson_plan	8
lesson_plan_session	9
student_registration	7
time_slot	7
timetable_clash	7
timetable_entry	12
timetable_version	10
topic_progress	7

admissions (8 tables)

Table	Cols
application	25
application_document	17
counselling_session	9
cutoff_history	10
enquiry	17
enquiry_interaction	8
enrolment	5
seat_allotment	6

agentops (16 tables)

Table	Cols
agent	19
agent_feedback	7
agent_incident	14
agent_output	13
agent_run	17
agent_run_input	7
agent_tool	7

Table	Cols
alert	14
fairness_audit	15
human_review	9
intervention	17
intervention_outcome	10
intervention_participant	3
model_version	20
risk_flag	26
schedule	9

assessment (15 tables)

Table	Cols
assessment	15
assessment_type	7
backlog	9
course_result	20
internal_mark	15
mark_anomaly	10
mark_change_log	9
paper_blueprint	12
paper_quality_review	9
paper_question	13
question_bank_item	21
question_paper	20
student_assessment_mark	13
student_question_mark	5
term_result	10

attendance (7 tables)

Table	Cols
attendance_record	10
attendance_record_2026	10
attendance_record_2027	10
attendance_summary	17
condonation	13
ingestion_batch	12

Table	Cols
student_leave	14

confidential (4 tables)

Table	Cols
academic_accommodation	8
counselling_case	13
counselling_note	6
crisis_escalation	10

core (10 tables)

Table	Cols
academic_year	10
building	4
calendar_event	8
campus	5
code_list	4
code_value	7
department	13
institution	12
room	14
term	13

curriculum (16 tables)

Table	Cols
batch	7
co_po_map	8
course	6
course_book	11
course_outcome	10
course_prerequisite	3
course_topic	6
course_unit	6
course_version	27
lab_batch	4
programme	16
programme_outcome	7
regulation	15

Table	Cols
regulation_norm	6
section	7
unit_outcome_map	2

engagement (10 tables)

Table	Cols
event	19
event_participant	15
event_submission	9
industry_activity	11
industry_partner	12
mou	17
mou_deliverable	8
outreach_activity	20
outreach_participant	4
review_assignment	11

exams (10 tables)

Table	Cols
answer_script	6
evaluation	6
exam_event	7
exam_registration	9
exam_room_allocation	6
exam_schedule	8
invigilation_duty	5
malpractice_case	10
reevaluation_request	10
seating_allocation	5

finance (15 tables)

Table	Cols
fee_demand	16
fee_demand_line	7
fee_head	7
fee_structure	14
fee_structure_line	3

Table	Cols
installment_plan	7
loan_document_request	11
payment	15
payment_allocation	3
refund	10
reminder_dispatch	12
scholarship_application	16
scholarship_eligibility	7
scholarship_renewal_risk	8
scholarship_scheme	15

governance (14 tables)

Table	Cols
circular	19
circular_recipient	6
committee	17
committee_compliance	9
committee_member	9
compliance_check	12
compliance_framework	5
compliance_requirement	11
meeting	15
meeting_attendance	3
meeting_item	7
policy_clause	7
policy_conflict	7
policy_document	17

hr (11 tables)

Table	Cols
admin_role_assignment	7
appraisal_cycle	8
appraisal_record	18
class_substitution	8
development_plan	11
faculty_attendance	8

Table	Cols
faculty_leave	17
faculty_workload	14
leave_balance	6
leave_type	8
workload_norm	9

identity (11 tables)

Table	Cols
app_user	12
audit_log	15
audit_log_2026	15
audit_log_2027	15
consent	7
data_request	9
delegation	7
permission	5
role	5
role_permission	2
user_role	12

knowledge (5 tables)

Table	Cols
chunk_embedding	6
document	22
document_chunk	7
extracted_field	12
extraction_job	9

outcomes (6 tables)

Table	Cols
attainment_action	9
attainment_rubric	11
attainment_run	10
co_attainment	16
indirect_feedback	6
po_attainment	15

people (8 tables)

Table	Cols
faculty	18
faculty_expertise	5
guardian	10
person	21
person_identifier	5
staff	8
student	18
student_lab_batch	2

placement (10 tables)

Table	Cols
alumni	21
alumni_contribution	7
company	11
drive_application	8
internship	23
internship_evaluation	9
job_opening	20
offer	15
readiness_assessment	8
readiness_summary	8

quality (7 tables)

Table	Cols
accreditation_criterion	8
criterion_readiness	10
evidence_item	15
feedback_instrument	11
feedback_response	7
kpi_definition	20
kpi_value	20

research (20 tables)

Table	Cols
affiliation_variant	4
call_faculty_match	7

Table	Cols
citation_snapshot	9
collaboration	11
funding_agency	5
funding_call	13
patent	18
patent_deadline	7
patent_inventor	6
phd_milestone	9
phd_scholar	18
project	16
project_member	6
proposal	10
publication	20
publication_author	9
publication_source	8
researcher_profile	8
venue	11
venue_metric	13

studentlife (10 tables)

Table	Cols
achievement	18
action_item	11
certification_catalog	11
disciplinary_case	19
disciplinary_step	8
grievance	21
grievance_event	6
mentor_meeting	12
mentorship	7
student_certification	13